UAV Software Engineering: ROS 2 Architecture

Workspaces, Colcon, and Ament

1 The Concept of a Workspace

In standard C++ development, a project typically resides in a single directory with one root `CMakeLists.txt`. In robotics, a UAV software stack is composed of dozens of independent packages (e.g., `camera_driver`, `flight_controller`, `telemetry_node`).

ROS 2 organizes these packages into a **Workspace**. A standard workspace contains three primary directories:

- `src/`: The only directory you will manually edit. It contains the source code for all your individual packages.
- `build/`: Where intermediate files (object files, CMake cache) are stored for each package.
- `install/`: Where the final compiled executables, libraries, and configuration scripts are placed.

2 Colcon: The Meta-Build System

To compile a workspace, ROS 2 uses **Colcon** (COLlective CONstruction). Instead of manually running `cmake` and `make` in each package, you run `colcon build` at the root of the workspace. Colcon analyzes the dependency graph of all packages in the `src/` folder, determines the correct build order, and invokes CMake for each one concurrently.

3 Package Anatomy: `package.xml` and `CMakeLists.txt`

For Colcon to recognize a folder inside `src/` as a ROS 2 package, it must contain two files:

1. `package.xml`: An XML manifest defining the package's name, version, license, and its dependencies (e.g., `rclcpp`, `std_msgs`). This is crucial for resolving dependencies before the build process begins.
2. `CMakeLists.txt`: The build script. However, ROS 2 extends standard CMake with the `ament_cmake` macro ecosystem. Ament simplifies finding ROS 2 dependencies and installing the generated executables into the workspace's `install/` directory.

4 The Overlay Environment (`setup.bash`)

Linux needs to know where to find your newly compiled UAV executables and libraries. When Colcon finishes building, it generates a `setup.bash` file inside the `install/` directory.

By executing `source install/setup.bash`, you prepend your workspace's paths to your system's environment variables (like `PATH` and `LD_LIBRARY_PATH`). This process creates an **Overlay** on top of the base ROS 2 installation (the *Underlay*), allowing the terminal to dynamically discover and run your custom drone nodes.

5 Advanced Colcon Strategies: Symlinks and Build Types

Building a massive UAV monorepo can take minutes or even hours. To optimize the development cycle, engineers rely on specific Colcon arguments:

- `--symlink-install`: By default, Colcon copies files from `src/` to `install/`. If you modify a Python script, a launch file, or a YAML configuration, you are forced to rebuild. Using `--symlink-install` creates symbolic links instead of copying. This allows changes to interpreted scripts and config files to take effect immediately without recompiling the workspace.
- **CMake Build Types:** By default, CMake compiles without heavy optimizations. In drone software, running unoptimized control loops can cause the CPU to miss real-time deadlines, leading to catastrophic crashes. It is mandatory to compile flight-ready nodes with hardware-specific optimizations using:
  `colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release`.

6 The Power of `ament_cmake` and Transitive Dependencies

Standard CMake requires developers to manually track and link every library and its dependencies using `target_link_libraries` and `target_include_directories`. In a complex robotics stack, this becomes unmaintainable.

ROS 2 solves this with `ament_cmake`. When you use `ament_target_dependencies(my_node rclcpp std_msgs)`, Ament automatically resolves all *transitive dependencies* (the dependencies of your dependencies). Furthermore, when you export your package using `ament_export_dependencies()`, any other node in the UAV stack that depends on your package will automatically inherit the correct include paths and linked libraries, ensuring a modular and robust build graph.

7 Environment Chaining: Underlays and Overlays

Professional drone software stacks are often containerized (as we will see in Chapter 7). Understanding how workspaces chain together is vital. The system installation of ROS 2 (e.g., in `/opt/ros/humble`) is the **Underlay**. When you source its `setup.bash`, your environment is populated with the base ROS 2 libraries.

When you compile your custom `uav_ws` and source its `setup.bash`, it becomes an **Overlay**. The overlay intelligently prepends its paths to the environment without overwriting the underlay. This allows you to have multiple layers of workspaces (e.g., a base ROS 2 underlay, a third-party driver middle-layer, and your custom flight controller overlay), enabling seamless integration of proprietary and open-source components.

8 Practical Workflow: Workspace Initialization

The following terminal commands represent the standard workflow to create a workspace, generate a package, and compile the overlay:

```
# 1. Create the workspace and the source directory
mkdir -p uav_ws/src
cd uav_ws/src

# 2. Source the system Underlay to enable ROS 2 CLI tools
# (Replace 'jazzy' with your specific ROS 2 distribution)
source /opt/ros/jazzy/setup.bash

# 3. Create a C++ package using ament_cmake
ros2 pkg create --build-type ament_cmake dummy_uav_pkg

# 4. Return to the workspace root and build using Colcon
cd ..
colcon build

# 5. Source the newly built Overlay to make the package visible to Linux
source install/setup.bash
```

9 Advanced Build Optimization

In production UAV environments, building can be optimized using:

- `--symlink-install`: Creates symbolic links instead of copying files, allowing Python scripts and config files to update immediately without recompiling.
- `Release Mode`: Drone software must meet hard real-time deadlines. Compile with hardware optimizations using:
`colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release`.